

# Results

Fitz Koch

2026-04-02

## Setup

```
import sys
sys.path.insert(0, "src")
```

## Imports

```
import importlib
import src.viz as viz

import constants as c
import numpy as np
from pathlib import Path
import matplotlib.pyplot as plt
import seaborn as sns

importlib.reload(viz)
from src.viz import viz_saved_sim
```

## Results

Currently, I've only run the 'restricted' breeding strategy with the 'name' sensing information. We don't get quite as good results as they do in the paper – they get down to **1.33** for name based and restricted breeding, but we only down to about **1.8** or so. It's very possible this is due to some small bug or misimplementation; certainly, we can see the results in an interesting fashion.

## Animations

```
viz_saved_sim("data/0/2026-04-02_11-30.npy", "Generation 0")
```

<IPython.core.display.HTML object>

```
viz_saved_sim("data/2/2026-04-02_11-39.npy", "Generation 2")
```

<IPython.core.display.HTML object>

```
viz_saved_sim("data/17/2026-04-02_11-54.npy", "Generation 17")
```

<IPython.core.display.HTML object>

```
viz_saved_sim("data/50/2026-04-02_12-01.npy", "Generation 50")
```

<IPython.core.display.HTML object>

## Results

Currently just testing out the code for the results. Running on extremely small datasets for now.

```
def plot_loss_curves(clone_data, free_data, restricted_data, title):
    breeding_strategies = ["Clone", "Free", "Restricted"]
    data_list = [clone_data, free_data, restricted_data]
    colors = {"Clone": "steelblue", "Free": "darkorange", "Restricted": "seagreen"}
    n = len(clone_data["avg_loss"])
    fig, axes = plt.subplots(nrows=1, ncols=2)
    avg_ax, best_ax = axes

    for ax in axes:
        ax.set(ylim=(0, 6))

    for strategy, data in zip(breeding_strategies, data_list):
        avg_ax.plot(range(n), data["avg_loss"], label=strategy, color=colors[strategy])
        best_ax.plot(
```

```

        range(n), data["best_loss"], label=strategy, color=colors[strategy]
    )

    avg_ax.legend()
    best_ax.legend()
    avg_ax.set_title("Average Loss")
    best_ax.set_title("Best Loss")
    plt.title(title)
    return fig

```

```

sensing_terminals = ["Name", "Deitic", "Base"]
"results/Restricted/Name/2026-04-02_checkpoint.npz"
timestamp = "2026-04-02"

```

```

for terminal in sensing_terminals:
    clone_data = np.load(Path("results") / terminal / "Clone" / f"{timestamp}_checkpoint.npz")
    free_data = np.load(Path("results") / terminal / "Free" / f"{timestamp}_checkpoint.npz")
    restricted_data = np.load(Path("results") / terminal / "Restricted" / f"{timestamp}_checkpoint.npz")
    plot_loss_curves(clone_data, free_data, restricted_data, title=terminal)

```



